ENSF 612 Project Proposal (Milestone 1, Revised)

Engineering Large-Scale Data Analytics Systems, Fall 2026
Instructor: Dr. Ebrahim Karami
October 25, 2026

1 Project Title

DupHunter: Scalable Duplicate Question Detection on Stack Overflow

2 Group Members

Name	Main role
Satyam Bhanot	Data pipeline, Spark cluster setup, scaling experiments
Iesrat Jahan	Duplicate detection core: shingling, MinHash, LSH, LSH settings study
Md Jubayer Khan	Baselines and accuracy evaluation; SON tag blocking
Navjot Singh Shienh	AI-impact analysis on duplicates; streaming duplicate check

3 Problem Statement

Stack Overflow is a website where programmers ask and answer questions. Many new questions have already been asked before, just in different words. These are called *duplicates*. Today, volunteers find and close them by hand, and many are missed.

We want to build a system that finds duplicate questions automatically. The hard part is size. Stack Overflow has about 24 million questions, and comparing every question with every other one would take years on one computer. We need methods built for large data that can run on more than one machine.

Duplicate detection is the core of this project. Every other part either improves it or uses its results.

Questions we want to answer:

Q1. **(Core)** How many known duplicates can MinHash and LSH find, and how much comparison work does it save compared with checking every pair?
Q2. **(Core)** How does the system scale with more cores, a second machine, and more data, and what slows it down?
Q3. Does grouping questions by frequent tag sets before LSH make it faster without losing many duplicates?
Q4. Did the share of new questions that are near-duplicates change after AI tools such as GitHub Copilot and ChatGPT became popular in 2022?

Expected outcome: a working Spark pipeline for duplicate detection, with measured accuracy and speed, plus two smaller studies built on top of it.

4 Motivation and Application

Duplicate questions waste time. People wait for answers that already exist, experts answer the same thing again, and volunteers spend hours searching. A tool that finds duplicates could show a user an existing answer before they post. The same methods are used to remove repeated pages from web search and from AI training data.

AI tools also changed how people use Stack Overflow. A 2024 study found that activity dropped after ChatGPT was released [3]. We want to see whether the questions that are still asked have become more or less repetitive.

The project fits ENSF 612 because it needs the course's tools for large data: MapReduce and Spark, similarity search, frequent itemsets, and stream algorithms [1].

5 Dataset

Source. We will use the public Stack Exchange data dump on the Internet Archive (`https://archive.org/details/stackexchange`). The newest release is from September 30, 2026 [2]. It is free to use under the CC BY-SA 4.0 license.

Size and format. The Stack Overflow archive is a single 64.2 GiB (68.9 GB) compressed file (`.7z`)

covering July 2008 to September 2026. Inside, each table is an XML file with one record per line. We only need the two tables in Table 1. Record counts are estimates from older releases; we will update them after loading the data.

Table 1: The data we will use.

Table	Records (approx.)	Fields we keep	What we use it for
Posts	60 million (24 million questions)	id, type, title, body, tags, date, author	Question text for the subset; dates and tags for all questions
PostLinks	10 million	post id, related post id, link type	Known duplicate pairs (our answer key)
Super User site	1.4 GB compressed	same tables	Testing our code

How much data we will use.

- We keep only the columns listed above and skip every other table.
- Duplicate detection needs the full question text and many test runs, so we run it on questions tagged `python` (roughly 2 million; we will confirm the count). This is still large: 2 million questions make about 2 trillion pairs. We only score duplicate pairs where both questions are in the subset.
- Dates and tags for all 24 million questions are small. We use them for overall question volume in Q4.
- We build and debug everything on the Super User site first. It has the same format but is about 50 times smaller.

Data quality and limitations.

- Question text is stored as HTML, with code mixed into the writing.
- Deleted posts are missing.
- Not every duplicate was marked by volunteers, so our answer key is incomplete and our precision will be a lower bound.
- The most recent months are incomplete, because new questions may still be closed as duplicates.
- Results on one tag may not hold for every topic.

Preprocessing. We extract only the two tables and read them as a stream, so we never store hundreds of GB of unpacked XML. We keep the needed columns and save them as Parquet (a compressed table format that Spark reads quickly), split by year. For the subset, we remove HTML tags, keep code blocks separate, and break each question into short groups of words called *shingles*.

6 Proposed Methodology

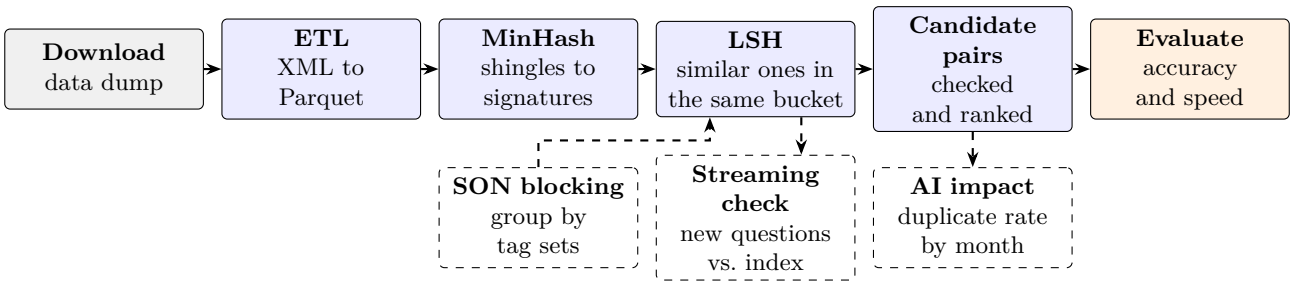


Figure 1: DupHunter pipeline. The top row is the core. Dashed boxes are the secondary parts.

Core: duplicate detection (Q1). We turn each question into shingles and compress them into a short MinHash *signature*. LSH then puts similar signatures into the same bucket, and we only compare questions that share a bucket. We will try several LSH settings (numbers of bands and rows), because too few bands miss duplicates and too many give false matches.

Secondary parts, all built on the core.

- **SON tag blocking (Q3).** Find frequent tag sets with the SON algorithm, then run LSH separately inside each tag group. We measure how much faster this is and how many duplicates it loses.
- **AI impact (Q4).** Use the duplicates found by the core to compute the monthly share of new questions that are near-duplicates, and compare it before and after June 2022 (Copilot) and November

2022 (ChatGPT).

- **Streaming check.** Replay questions in date order and check each new one against the LSH index as it arrives. DGIM keeps a rolling count of duplicates in the most recent questions, and Flajolet-Martin counts distinct askers. Unlike an exact Spark SQL query, which needs all the data stored and more memory as data grows, these answer in one pass with fixed memory. We will show their error for different memory sizes, compare their memory with the exact counts, and measure updates per second.

Priority and cut order. Q1 and Q2 must be finished. If time runs short, we cut the streaming check first, then the AI-impact study, then SON blocking. The core on its own still covers every guideline requirement.

Compute setup. We have two laptops, each with an Intel Core i7 and 16 GB of RAM. We will connect them as a Spark standalone cluster [5]: Laptop A runs the master, the driver, and one worker; Laptop B runs one worker. Each worker gets about 8 cores and 10 GB of memory. MinHash signatures for 2 million questions need only about 1 GB, so this is enough. Both laptops will be on the same wired network, so that moving data between them does not dominate the timings. The laptops also have NVIDIA GPUs, which we will not use, because the course algorithms do not need them.

How we will evaluate.

- **Accuracy (Q1):** Recall@K, Precision@K, MAP (how high real duplicates rank), and reduction ratio (the share of pairs we never compare). Baselines: an exact all-pairs check on a sample of 100,000 questions, and Spark's built-in `MinHashLSH`.
- **Blocking (Q3):** run time, data moved between machines, and recall, with and without SON blocking.
- **AI impact (Q4):** monthly duplicate share and question volume, before and after 2022.
- **Scaling (Q2):** the three tests in Table 2. For each run we record run time, speedup (how many times faster it is than with one core), data moved between machines, and peak memory. Each setting runs three times.

Table 2: Scaling tests on the two-laptop cluster.

Test	Setup	What it shows
A. More cores	1, 2, 4, and 8 cores on one laptop; same data	Speedup from adding cores
B. Second machine	8 cores on one laptop vs. 4 + 4 on two laptops; then 8 + 8	The cost of sending data over the network
C. More data	10%, 25%, 50%, and 100% of the subset on both laptops	How run time grows with data size

7 Algorithms and Techniques

We write the course algorithms ourselves and compare them with ready-made tools, so the report clearly shows which code is ours.

Table 3: Algorithms, baselines, and priority.

Our own code	Compared with	Priority	Q
Shingling, MinHash, LSH [6, 7] in Spark	Exact check on a sample; Spark `MinHashLSH`	Core	Q1
Spark jobs in the MapReduce style [4]	Same jobs with fewer cores or one machine	Core	Q2
SON frequent itemsets [8] for tag blocking	LSH without blocking; Spark `FPGrowth`	Secondary	Q3
Monthly before/after comparison of duplicate share	Same months before 2022	Secondary	Q4
DGIM [9], Flajolet-Martin [10] in a streaming check	Exact counts in Spark SQL	Cut first	Q1

8 Expected Challenges

Challenge	How we plan to handle it
Duplicates written in very different words	Measure how recall changes with similarity; note it as a limit of word-based shingles
Missing duplicate labels	Treat precision as a lower bound; rely mainly on recall
Some LSH buckets become huge (common phrases)	Limit bucket size and split large buckets
Choosing LSH settings	Test several band and row settings on Super User first
Moving data between the two laptops	Wired network; process LSH buckets in batches; measure data moved
16 GB of memory per laptop	Keep only needed columns; keep large results on disk; use the subset
Blocking misses duplicates with different tags	Measure the recall lost compared with no blocking

9 Expected Results

- **Q1:** LSH will find most known duplicates while skipping more than 99.9% of all question pairs, and our version will perform close to Spark's built-in one.
- **Q2:** adding cores will help most for MinHash, which needs no communication. The second laptop will help less for LSH, because data must move over the network.
- **Q3:** tag blocking will cut run time and data moved, with a small loss in recall.
- **Q4:** the share of near-duplicate questions will change after 2022; we expect a drop in simple, repeated questions.

10 Initial Project Schedule

The core comes first. On November 13 we check progress and decide whether the secondary parts go ahead.

Table 4: Weekly plan (blue: main work; orange: milestone).

Task (week starting)	Who	Oct 26	Nov 2	Nov 9	Nov 16	Nov 23	Nov 30
Download, ETL, cluster setup	Satyam						
MinHash and LSH, settings study	Iesrat						
Baselines and accuracy evaluation	Jubayer						
Core checkpoint	All			Nov 13			
Scaling tests	Satyam						
SON tag blocking	Jubayer						
AI impact; streaming check	Navjot						
Results freeze	All				Nov 20		
Presentation	All						
Final report and code	All						Dec 4

11 References

[1] A. Rajaraman and J. D. Ullman, *Mining of Massive Datasets*. Cambridge University Press, 2011.

[2] Stack Exchange, Inc., "Stack Exchange Data Dump 2026-09-30," Internet Archive, https://archive.org/details/stackexchange_20260930.

[3] R. M. del Rio-Chanona, N. Laurentsyeva, and J. Wachs, "Large language models reduce public knowledge sharing on online Q&A platforms," *PNAS Nexus*, vol. 3, no. 9, 2024.

[4] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," in *Proc. OSDI*, 2004.

[5] M. Zaharia *et al.*, "Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing," in *Proc. NSDI*, 2012.

[6] A. Z. Broder, "On the resemblance and containment of documents," in *Proc. Compression and Complexity of Sequences*, 1997.

[7] P. Indyk and R. Motwani, "Approximate nearest neighbors: Towards removing the curse of dimensionality," in *Proc. ACM STOC*, 1998.

[8] A. Savasere, E. Omiecinski, and S. Navathe, "An efficient algorithm for mining association rules in large databases," in *Proc. VLDB*, 1995.

[9] M. Datar, A. Gionis, P. Indyk, and R. Motwani, "Maintaining stream statistics over sliding windows," *SIAM J. Comput.*, vol. 31, no. 6, 2002.

[10] P. Flajolet and G. N. Martin, "Probabilistic counting algorithms for data base applications," *J. Comput. Syst. Sci.*, vol. 31, no. 2, 1985.